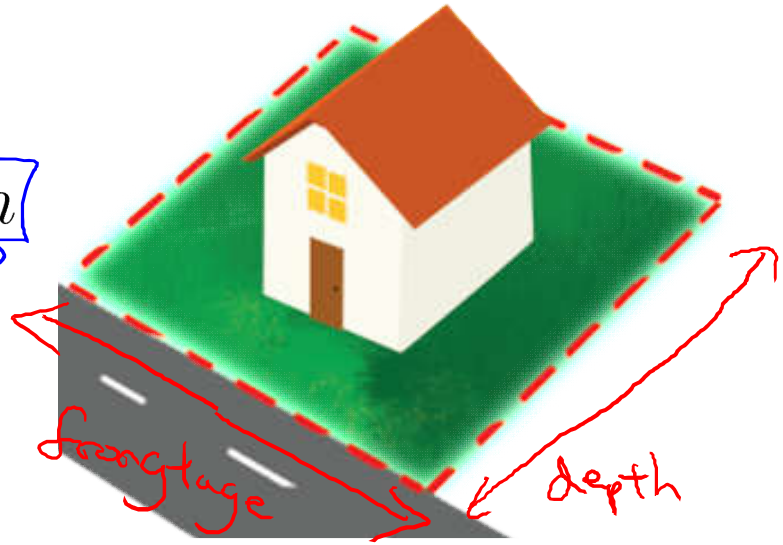
Housing prices prediction

$$h_\theta(x) = \theta_0 + \theta_1 \times frontage + \theta_2 \times depth$$

x_1 x_2



Area

x = frontage * depth

$h_\theta(x) = \theta_0 + \theta_1 x$

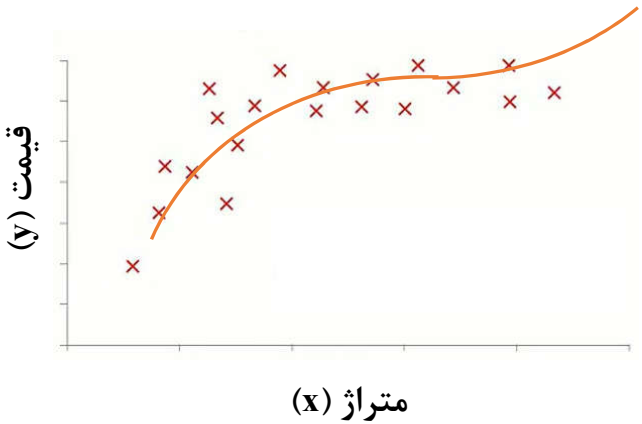
↑ land area

برازش چندجمله

❖ **برازش چندجمله‌ای** (Polynomial Regression) برای پیشبینی متغیر وابسته‌ای که به صورت چندجمله‌ای با متغیرهای مستقل مرتبط است به کار می‌رود.

مثال

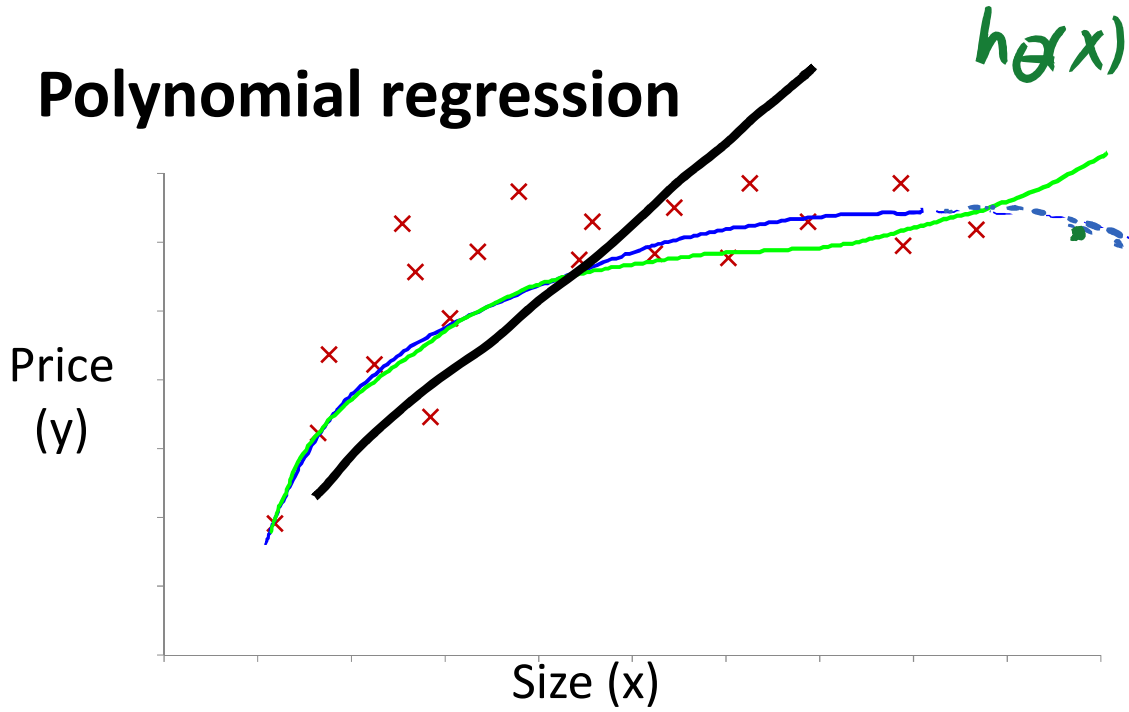
برازش چندجمله‌ای: پیش‌بینی قیمت خانه



$$h_\theta(x) = \theta_0 + \theta_1(size)^1 + \theta_2(size)^2 + \theta_3(size)^3$$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

Polynomial regression



$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2$

$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$
$$= \theta_0 + \theta_1 (size) + \theta_2 (size)^2 + \theta_3 (size)^3$$

$x_1 = (size)$

$x_2 = (size)^2$

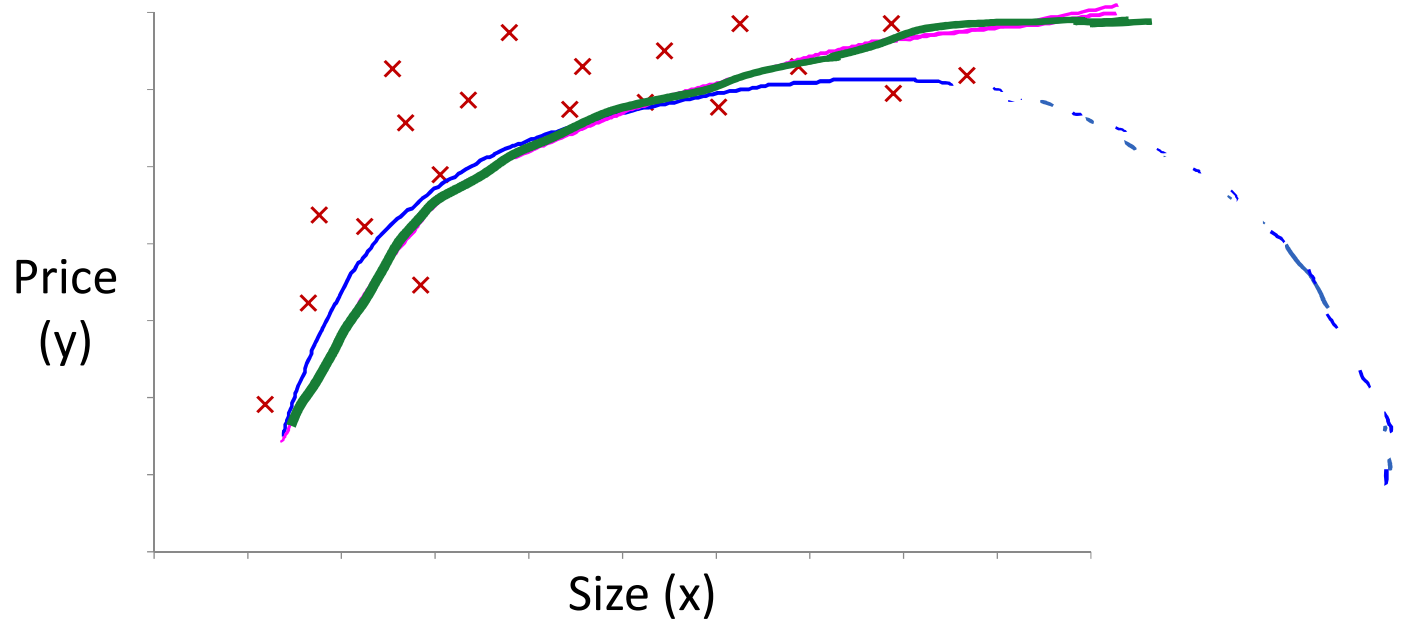
$x_3 = (size)^3$

Size: 1 - 1000

Size^2: 1 - 1000,000

Size^3: 1 - 10^9

Choice of features

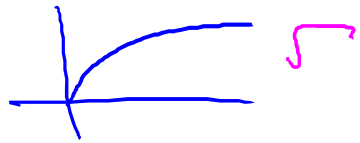


x: size

$$h_\theta(x) = \theta_0 + \theta_1(size) + \theta_2(size)^2$$

$$h_\theta(x) = \theta_0 + \theta_1(size) + \theta_2\sqrt{(size)}$$

$x_1 = x$ $\quad$ $x_2 = \sqrt{x}$



Stochastic Gradient Descent vs Batch Gradient Descent

Batch Gradient Descent

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

$$\theta_{t+1} = \theta_t - \alpha J'(\theta)$$

Need to compute for all data,
if you have sample size, m = 1million
Very slow to update θ

Stochastic Gradient Descent

$$cost\left(\theta, (x^{(i)}, y^{(i)})\right) = \frac{1}{2}(h_\theta(x^{(i)}) - y^{(i)})^2$$

1. Random Shuffle Data
2. Repeat {
   for i = 1, … , m {

$$\theta_{t+1} = \theta_t - \alpha \frac{\partial}{\partial \theta_t} cost(\theta, (x^{(i)}, y^{(i)}))$$

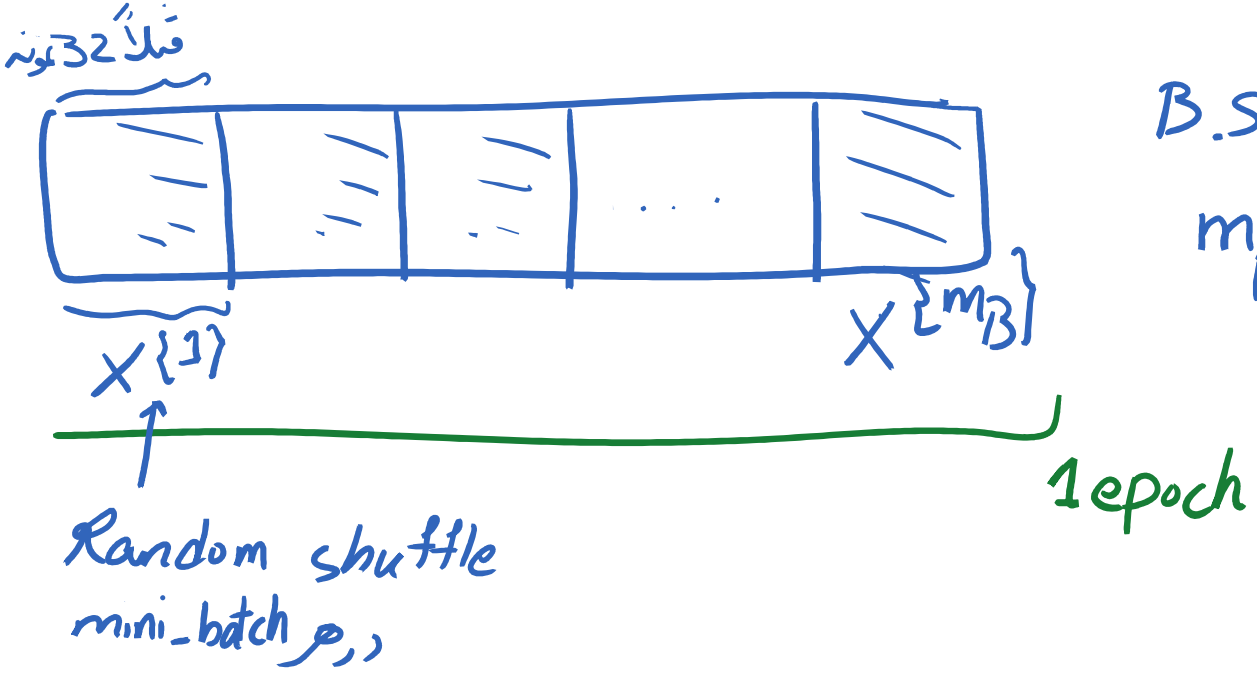
   }
}

Update θ using only one data point.
Faster

ممکن است حول Opt نوسان کند ← α ثابت نباشد

Mini-batch Gradient Descent

❖ اگر تعداد نمونه ها (m)، خیلی بزرگ باشد، به چندین **mini-batch** تقسیم می شوند و در هر مرحله یکی از **mini-batch**ها در به هنگام رسانی پارامترها استفاده می شود.



B.S : 32, 64, 128, . . .

$m_B = \frac{m}{B.S.}$

$\theta_0 = b$

$[\theta_1 \quad \theta_n]' = W$

Mini-batch Gradient Descent

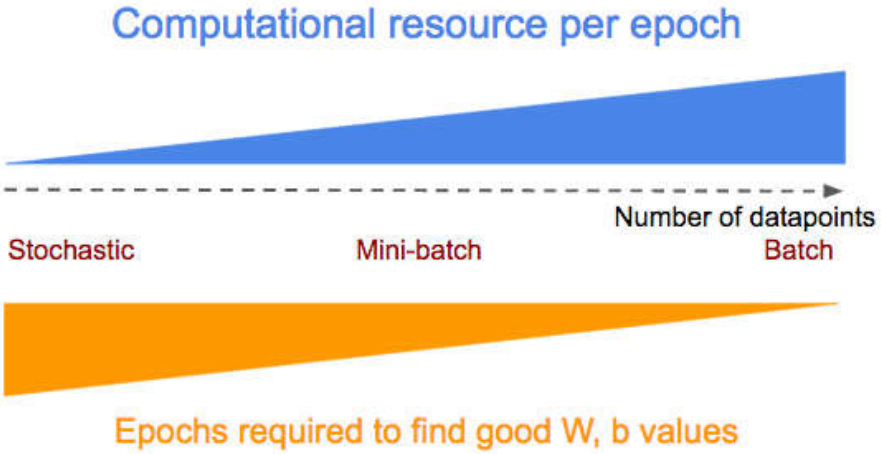
$\frac{m}{t} = m_B$

$$X = \begin{bmatrix} \vdots & \vdots & & \vdots \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ \vdots & \vdots & & \vdots \end{bmatrix}_{n_x \times m}$$

$t = 1 \rightarrow Stochastic$

$t = m \rightarrow Batch$

$t = 64, 128, 256, 512 \rightarrow Mini - batch$



for every t:

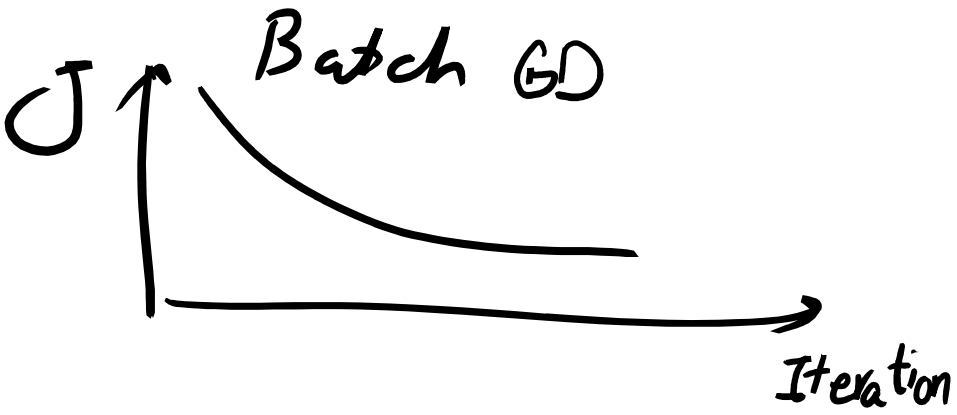
Optimize based on $X^{(1:t)}$

Compute *cost J*

Update θ

$X^{(1)} \dots X^{(64)} \mid X^{(65)} \dots X^{(128)} \mid \dots$

$X^{\{1\}}$ $X^{\{2\}}$



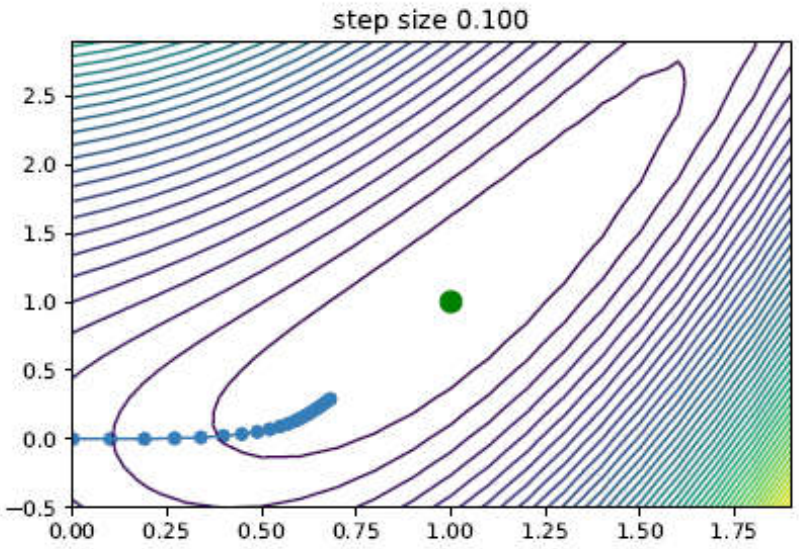


ممکن است یادگیری $X^{\{2\}}$ از $X^{\{1\}}$ مشکل‌تر باشد $\Leftarrow J^{\{2\}} > J^{\{1\}}$ *

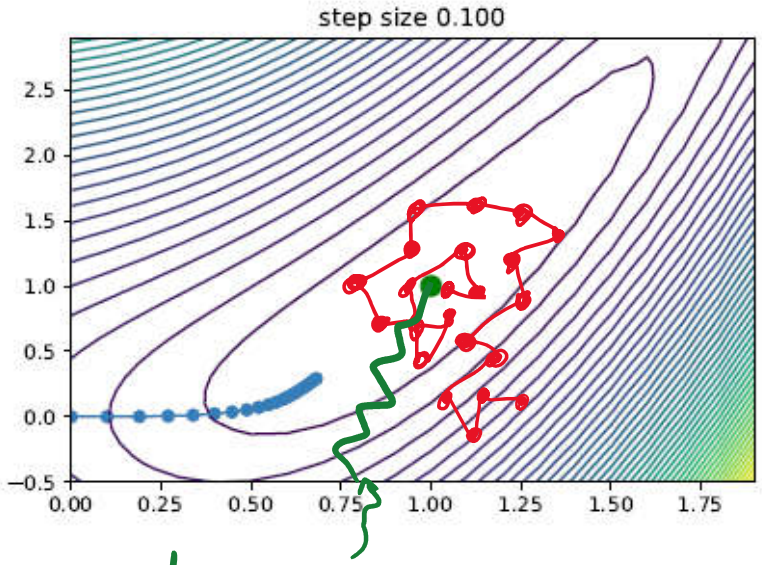
$1 < m_B < m$ $\quad m_B = m \rightarrow$ stochastic G.D.

$m_B = 1 \rightarrow$ Batch G.D.

Batch Gradient Descent



Stochastic Gradient Descent



$1 < m_B < m$

Fastest learning

Batch Gradient Descent

Too long per iteration especially when *m* is very large

Stochastic Gradient Descent

Lose speedup from vectorization (Inefficient implementation)

Newton Method

x_{init}

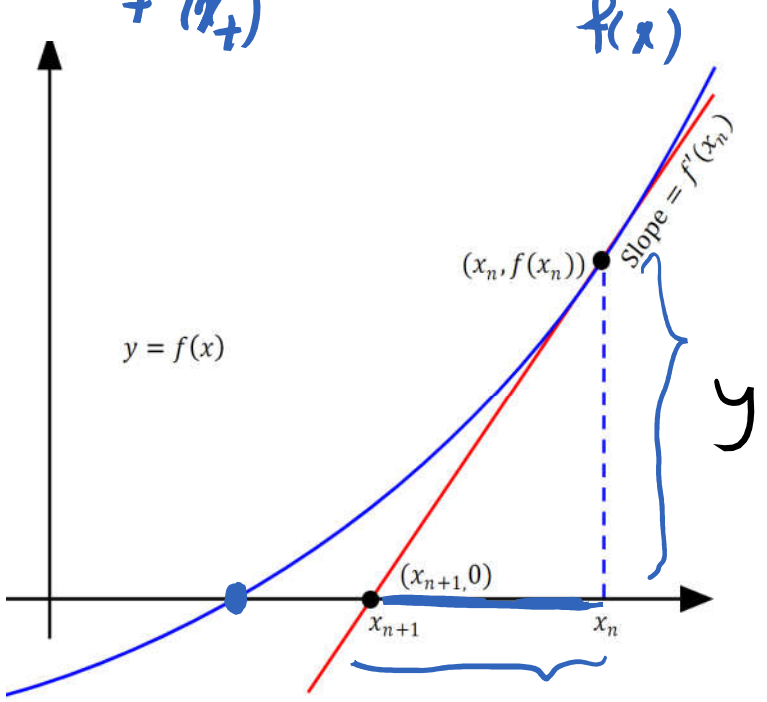
$x_{t+1} = x_t - \frac{f(x_t)}{f'(x_t)}$

ریشه‌یابی $f(x)$:

$J(\theta)$ $\quad J'(\theta) = 0$

θ_{init} $\quad \theta_{t+1} = \theta_t - \frac{J'(\theta_t)}{J''(\theta_t)}$

تک‌متغیره نیوتن



$\underline{\theta} = [\theta_0 \quad \theta_n]^T_{n+1 \times 1}$

$J(\underline{\theta})$ $\quad \nabla_\theta J(\theta_{opt}) = \left[\frac{\partial J}{\partial \theta_0}, \ldots, \frac{\partial J}{\partial \theta_n}\right]^T$

$\underline{\theta}_{init}$ $\quad \underline{\theta} := \underline{\theta} - H^{-1} \nabla_\theta J(\theta)$

نیوتن راپسون

$H_{(n+1)(n+1)}: H_{ij} = \frac{\partial J}{\partial \theta_i \theta_j}$

$\begin{bmatrix} \frac{\partial^2 J}{\partial \theta_0^2} & \cdots \\ \vdots & \ddots \\ & \frac{\partial^2 J}{\partial \theta_n^2} \end{bmatrix}$

- Faster convergence

Newton Method

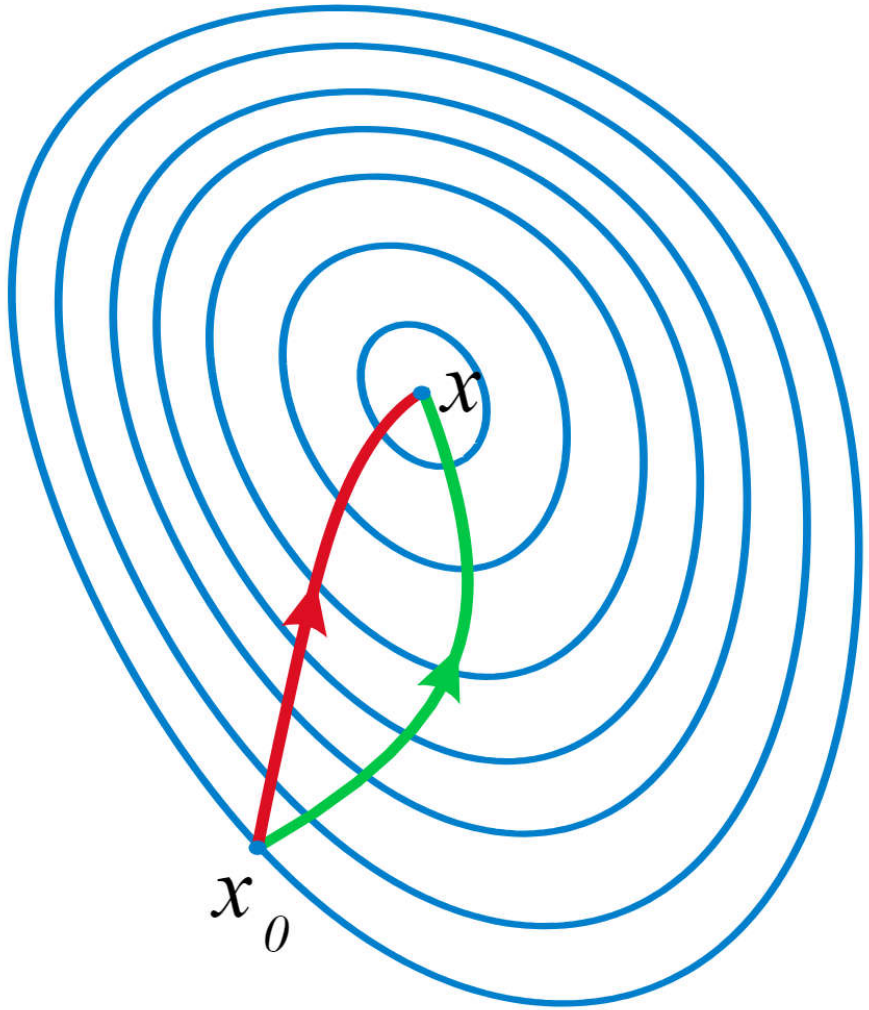
سرعت همگرایی نیوتن - رافسون می تواند بسیار سریع باشد.

ولی $H^{-1}_{(n+1)(n+1)}$ برای n بزرگ هزینه محاسباتی زیادی دارد.

وقتی روش نیوتن رافسون برای حل مسئله log. به کار می رود ← Reg

Fisher Scoring

Newton Method vs Gradient Descent



Logistic Regression

Introduction

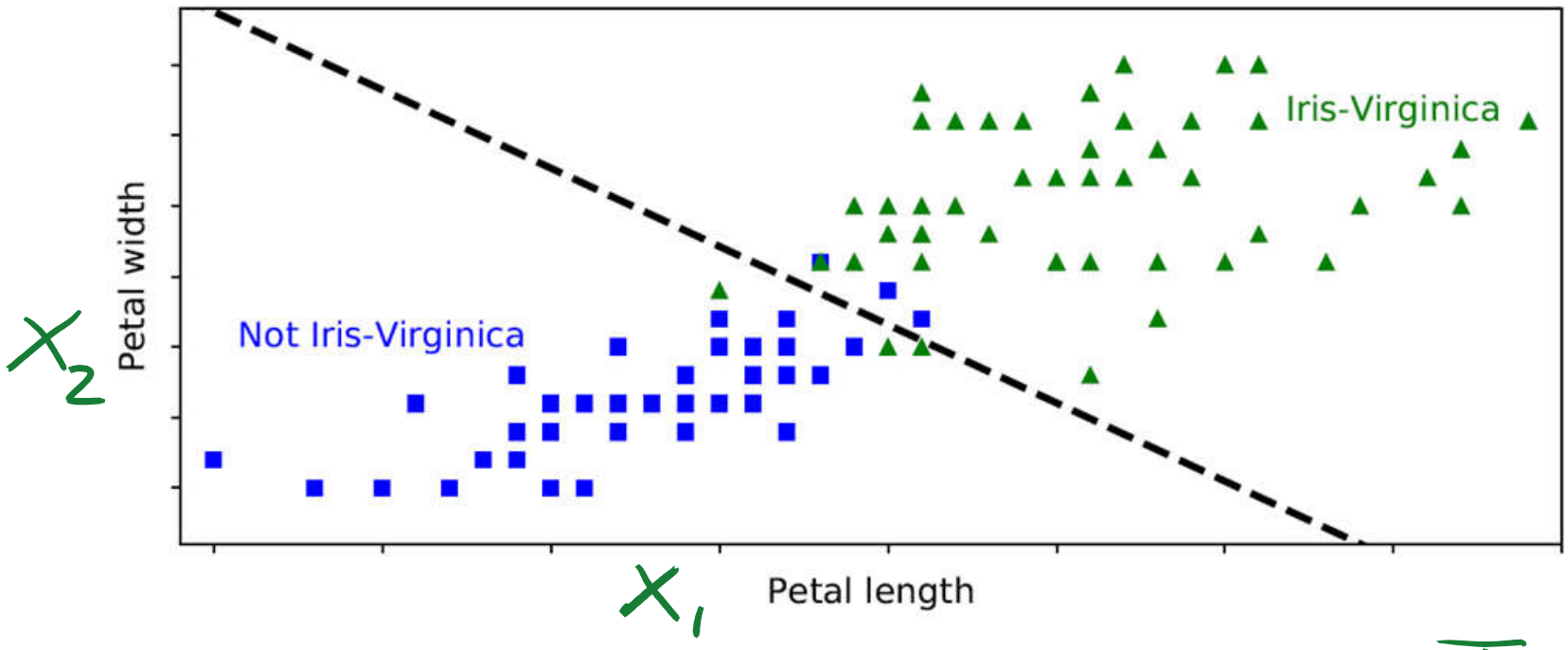
- Logistic regression is a widely used discriminative classification model $p(y \mid \mathbf{x}; \boldsymbol{\theta})$.
- where $\mathbf{x} \in \mathbb{R}^n$ is a fixed-dimensional input vector, $y \in \{1, \ldots, C\}$ is the class label.
- If $C = 2$, this is known as binary logistic regression.
- if $C > 2$, it is known as multinomial logistic regression, or alternatively, multiclass logistic regression.

Classification

Email: Spam / Not Spam?
Online Transactions: Fraudulent (Yes / No)?
Tumor: Malignant / Benign ?

$y \in \{0, 1\}$

0: “Negative Class” (e.g., benign tumor)
1: “Positive Class” (e.g., malignant tumor)



$h_\theta(x) = \theta^T x$

$-\infty \rightarrow \infty$

Threshold classifier output $h_\theta(x)$ at 0.5:

If $h_\theta(x) \geq 0.5$, predict "y = 1"

If $h_\theta(x) < 0.5$, predict "y = 0"

Classification: y = 0 or 1

$h_\theta(x) = \theta^T x$

$h_\theta(x)$ can be > 1 or < 0

Logistic Regression: $0 \leq h_\theta(x) \leq 1$

$h_\theta(x) = P(y=1 \mid x)$

طبقه‌بندی باینری

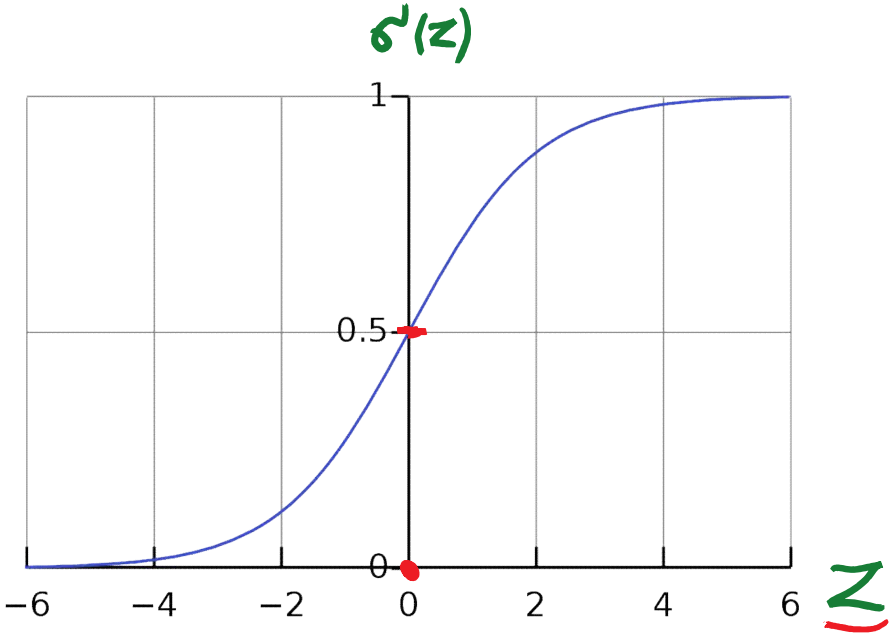
Logistic Regression Model

We want $0 \leq h_\theta(x) \leq 1$

$h_\theta(x) = g(\theta^T x)$

$g(z) = \frac{1}{1 + e^{-z}}$

$H_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$



Interpretation of Hypothesis Output

$\{x^{(i)}, y^{(i)}\}_{i=1}^{m}$ $y^{(i)} \in \{0, 1\}$

$h_\theta(x)$ = estimated probability that y = 1 conditioned on input x

Example: If $x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{tumorSize} \end{bmatrix}$

$h_\theta(x^{(i)}) = g(\theta^T x^{(i)})$

$x^{(i)} \longrightarrow y^{(i)}$

$h_\theta(x) = P(y=1 \mid x)$

$1 - h_\theta(x) = P(y=0 \mid x)$

$h_\theta(x)_{test} = 0.7$

$$P(y=0|x;\theta) + P(y=1|x;\theta) = 1$$
$$P(y=0|x;\theta) = 1 - P(y=1|x;\theta)$$

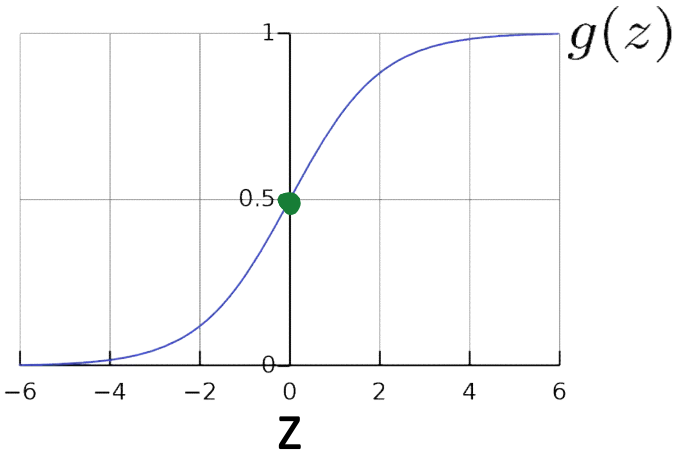
$h_\theta(x)_{test} = 0.3$

$x_{test} \rightarrow \dfrac{1}{1 + e^{-\theta^T x_{test}}} \gtrless^{th} 0.5$

Logistic regression

$$h_\theta(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$



Suppose predict "$y = 1$" if $h_\theta(x) \geq 0.5$

$\theta^T x = 0 \Rightarrow g(\theta^T x) = \frac{1}{2}$

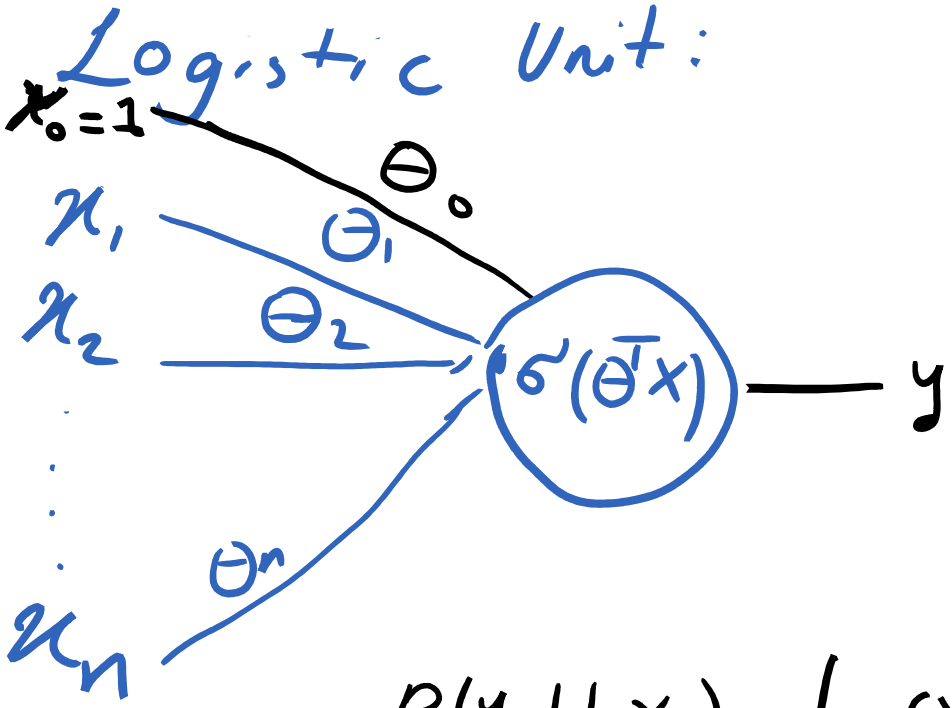
predict "$y = 0$" if $h_\theta(x) < 0.5$

$th = 0.5$: مرز تصمیم گیری

$\theta^T x \geq 0 \rightarrow y = 1$

$\theta^T x < 0 \rightarrow y = 0$

Logistic Unit:



$0 \leq h_\theta(x) \leq 1$

$$P_\theta(y=1 \mid x) = h_\theta(x) = \frac{1}{1+e^{-\theta^T x}} \geq 0.5$$

$$\theta^T x \geq 0$$

Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \cdots, (x^{(m)}, y^{(m)})\}$

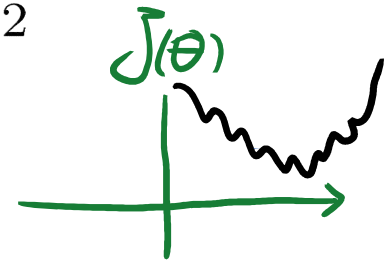
m examples $x \in \begin{bmatrix} x_0 \\ x_1 \\ \cdots \\ x_n \end{bmatrix}$ $x_0 = 1, y \in \{0, 1\}$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

How to choose parameters θ ?

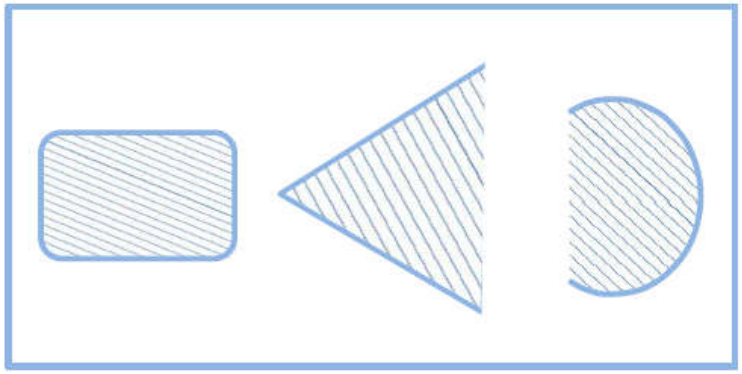
Cost function

Linear regression: $J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \frac{1}{2} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$

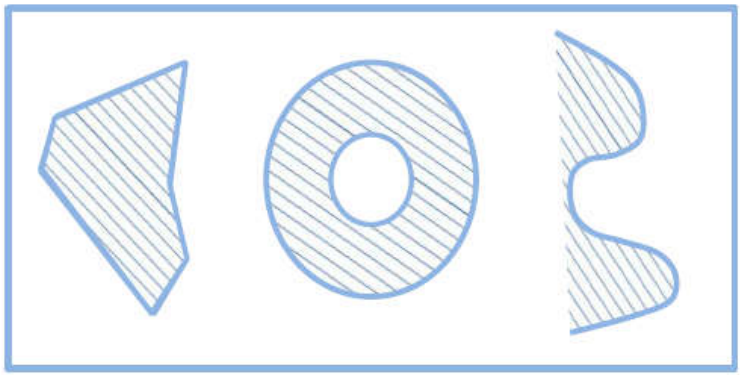


$\sigma(\theta^T x^{(i)})$

$\text{Cost}(h_\theta(x^{(i)}), y^{(i)}) = \frac{1}{2} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$ → غیرمحدب



Convex



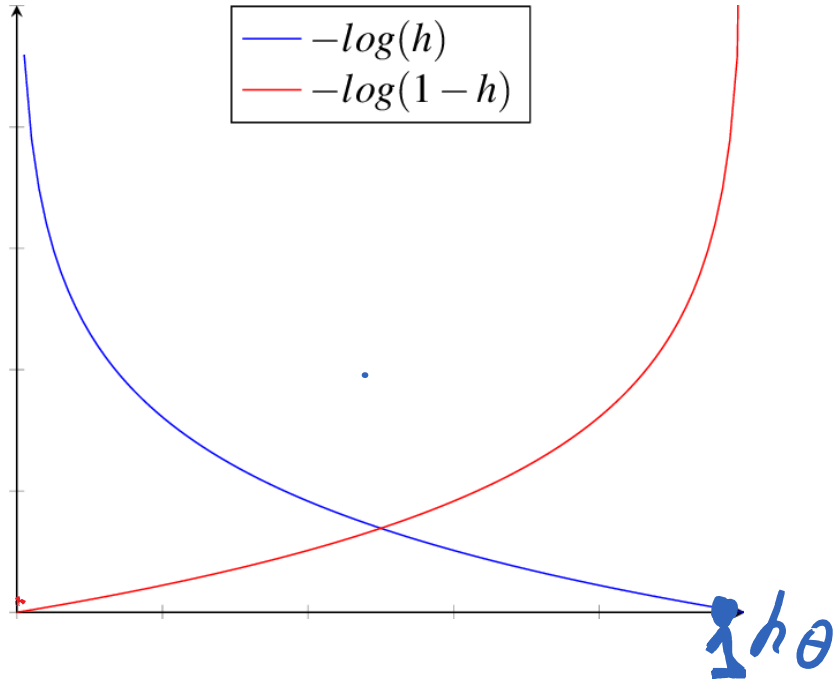
Not Convex

Logistic regression cost function

$h_\theta(x) = P(y=1 \mid x)$

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

Convex



Logistic regression cost function

$$J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

Note: $y = 0$ or 1 always

$$\text{Cost}(h_\theta(x), y) = -y \log(h_\theta(x)) - (1 - y) \log(1 - h_\theta(x))$$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Logistic regression cost function

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

$P(y=1|x)$ $P(y=0|x)$

Cross entropy تابع خطای

Gradient Descent

θ

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$\}$

(simultaneously update all θ_j)

$j = 0 \ldots n$